

南開大學

实验报告

实验 02：编写数据包捕获程序
并对捕获的数据包进行分析

姓名：尚文轩

学号：2412456

专业：计算机科学与技术

任课教师：徐敬东、张建忠

2026 年 10 月 22 日

目录

1 实验目的与要求对应 **2**

2 实验环境与 Npcap 架构 **2**
2.1 实际环境 . . . 2
2.2 架构与数据路径 . . . 3
2.3 链路类型不能混用 . . . 3

3 数据帧格式与解析规则 **4**
3.1 以太网必显字段 . . . 4
3.2 类型字段与长度字段的区分 . . . 4
3.3 VLAN 的两个不同“首字节” . . . 4
3.4 截断、字节序与解析范围 . . . 4

4 程序设计与实现 **5**
4.1 模块划分 . . . 5
4.2 API 调用顺序与含义 . . . 5
4.3 整体流程与退出机制 . . . 6
4.4 输出设计与可选保存 . . . 7

5 实验步骤与测试验证 **7**
5.1 构建与实时操作 . . . 7
5.2 纯解析器测试 . . . 8
5.3 真实 exe 的集成测试 . . . 8

6 真实捕获结果与逐帧分析 **9**
6.1 捕获配置与总体结果 . . . 9
6.2 题目必显字段的逐字节核对 . . . 9
6.3 第一帧的十六进制分析 . . . 10
6.4 请求与应答配对 . . . 10
6.5 程序界面输出 . . . 11

7 问题处理、结论与局限 **11**
7.1 主要问题及解决 . . . 11
7.2 实验结论 . . . 11

参考文献 **12**

1 实验目的与要求对应

本实验使用 C++17 与 Npcap，在 Windows 上枚举网卡、打开指定设备，捕获本机以太网视图下的数据帧，并在屏幕上显示源 MAC 地址、目的 MAC 地址、类型/长度字段与该字段之后第一个字节。通过对真实 ICMP 请求与应答帧的逐字节复核，理解链路层封装、网络字节序和捕获长度的含义；通过独立边界测试验证解析器不会在截断数据上越界。

表 1: 任务要求、实现与验证证据

要求	实现内容	证据
了解 Npcap 架构	用户态 API、DLL、NDIS 过滤驱动、适配器分层	第2节架构图
设备枚举与打开	findalldevs、create、配置、activate	设备列表与真实网卡测试
捕获并显示必需字段	next_ex、显式按字节解析以太网头	真实 PCAP 与屏幕输出逐字段一致
简明屏幕显示，无需保存	主表四个必显字段，默认无落盘	默认不写抓包文件测试
结构清晰，可读性好	解析、Npcap 资源、命令行与捕获流程分模块	模块表、核心代码、异常清理
源码、可执行程序与报告	C++、Windows x64 exe、LaTeX 与 PDF	独立实验目录和白名单提交包

2 实验环境与 Npcap 架构

2.1 实际环境

操作系统为 Windows 11（10.0.26200）；编译器为 MSVC 19.29 x64，编译选项包含 `/std:c++17/W4/WX/utf-8/O2/MT`，警告作为错误处理，静态链接 C/C++ 运行库。通过官方 Npcap SDK 1.16 的头文件获得 API 声明，运行时从系统 Npcap 目录加载 DLL。实际捕获环境如下表，数据来自本机实测记录。

表 2: 真实捕获的运行环境

Npcap	Npcap version 1.89, based on libpcap version 1.10.7 (64-bit time_t)
网卡	MediaTek Wi-Fi 6 MT7921 Wireless LAN Card
枚举编号	6（仅对本次枚举有效）
本机 IPv4	10.130.33.54
默认网关	10.130.0.1
过滤器	icmp and host 10.130.0.1

本机原有旧 Npcap 驱动，但缺少配套 DLL；第一次安装失败，第二次由用户确认官方安装窗口完成安装。后续以 DLL 实际版本、成功枚举网卡、打开设备及真实捕获验证环境，未将

“下载完成”或“安装器启动”视为安装成功。Npcap 安装时启用了管理员访问限制，因此真实捕获测试在用户确认的管理员进程中运行。

2.2 架构与数据路径

Npcap 提供 libpcap 风格的用户态 API，底层使用 Windows 内核驱动获取数据。其驱动作为 NDIS 6 轻量级过滤驱动连接网络栈和适配器；应用程序不直接读取网卡硬件寄存器，也不能用普通 TCP `recv` 代替链路层抓包 [1]。

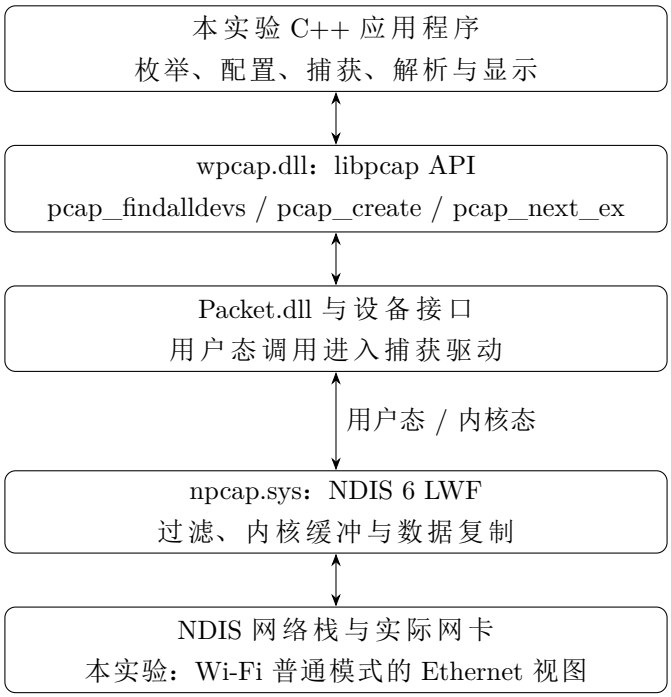


图 1: 本实验使用的 Npcap 捕获数据路径（概念图）

2.3 链路类型不能混用

`pcap_datalink` 给出本句柄数据的链路类型。普通 Ethernet 或 Wi-Fi 适配器在本实验中返回 `DLT_EN10MB=1`，可以按以太网头解析；这不意味着抓到了原始 802.11 无线管理帧。未启用无线监控模式，不把无线信道、重试标志或 Radiotap 等当成已捕获内容。

Npcap 回环通过 Windows Filtering Platform 支持本机流量，通常使用 `DLT_NULL`，只有地址族等伪首部，没有以太网源/目的 MAC[1]。因此程序对非 DLT_EN10MB 句柄直接提示错误并退出，不将回环 IP 数据的前 12 字节伪装为 MAC。离线 DLT_NULL 合成文件也用于验证该拒绝路径。

3 数据帧格式与解析规则

3.1 以太网必显字段

对 DLT_EN10MB，以下偏移均从 Npcap 返回的数据缓冲起点计数，不包含前导码和 SFD。MAC 按原始字节顺序输出，类型字段按大端解释。

表 3: 题目要求的字段与字节位置

字节偏移	长度	含义	读取条件
0–5	6	目的 MAC	caplen ≥ 14
6–11	6	源 MAC	caplen ≥ 14
12–13	2	类型/长度字段	caplen ≥ 14
14	1	字段之后第一个字节	caplen ≥ 15

程序把是否存在一个字段与字段的数值分开表示。例如 ARP 后首字节可能为 `0x00`，这仍是实际捕获到的有效数值，不能因为它等于零就显示 N/A。若捕获长度仅为 14，则 MAC 与类型字段有效，但首字节不存在；若不足 14，则整条标记为短以太网帧。

3.2 类型字段与长度字段的区分

IEEE 802.3 长度字段的最大值是 1500；字段值达到 1536（`0x0600`）及以上才按 EtherType 解释；1501–1535 是保留区间。本实验常见 EtherType 为 IPv4（`0x0800`）、ARP（`0x0806`）、IPv6（`0x86DD`）。不能将所有 16 位数值都称为 IPv4 类型，也不能把某一条样本的首字节固定写入程序。类型数值参照 IEEE 802 Numbers 登记表 [6]。

3.3 VLAN 的两个不同“首字节”

对于 `0x8100` 或 `0x88A8`，偏移 14 开始是 VLAN 控制字段 TCI，之后才是内层 EtherType 和实际负载。题目要求的“类型字段之后第一个字节”始终显示**外层字段之后的原始偏移 14**；程序另外显示内层类型、VLAN ID、负载偏移及负载首字节，避免混淆。

例如合成测试字节 `8100A064080045` 表示外层类型 `0x8100`，After 为 `0xA0`，TCI 为 `0xA064`，VLAN ID 为 100，内层类型为 IPv4，真正 IP 首字节为 `0x45`，负载从偏移 18 开始。双标签时继续向后解析；每次至少检查剩余四个字节，循环次数受实际捕获长度限制。此处 VLAN 是明确标记的**合成边界测试**，本次真实网卡样本并未捕获 VLAN 标签。

3.4 截断、字节序与解析范围

`pcap_pkthdr.caplen` 是缓冲区实际可读取的字节数，`len` 是原始帧长度。若 caplen 小于 len，只能在 caplen 范围内读取，并标注 TRUNCATED。不能因为原始帧足够长就读取未捕获

的内存。本程序不强转为带平台填充的头部结构体，使用显式字节操作；因此未对齐缓冲区也能安全解析。

```
inline uint16_t read_be16(const uint8_t* p) {
    return static_cast<uint16_t>(
        (static_cast<uint16_t>(p[0]) << 8) | p[1]);
}
// MAC destination: data[0..5]; source: data[6..11].
// Type/length: read_be16(data + 12).
// Mandatory first byte: data[14], only if caplen >= 15.
```

4 程序设计与实现

4.1 模块划分

文件 / 模块	职责
src/ethernet.hpp	无驱动依赖的帧解析、MAC 格式化、类型分类、VLAN 与可选 IP/ARP 解释
src/npcap_api.hpp	系统 Npcap DLL 加载、SDK 函数类型解析、设备列表/句柄/转储资源回收
src/main.cpp	参数校验、设备枚举、捕获流程、屏幕或 JSON 输出、退出和统计
tests/parser_tests.cpp	24 项纯 C++ 边界测试，无需 Npcap 运行库
tests/integration.py	驱动真实 exe，比较捕获与 PCAP 原始字节
tests/run-elevated.ps1	选择物理默认路由网卡，记录环境，在管理员进程运行测试
tests/prepare_report.py	从真实保存证据生成报告表格和十六进制记录

4.2 API 调用顺序与含义

先使用官方 SDK 的函数声明，通过 `LoadLibraryExW` 加载系统 Npcap 目录的 DLL，并用 `GetProcAddress` 解析函数。搜索目录限制为该 DLL 所在目录与系统目录，避免误用旧 WinPcap。`pcap_init(PCAP_CHAR_ENC_UTF_8)` 明确约定 Npcap 参数、返回字符串的编码；程序 `wmain` 将 Windows UTF-16 参数转换成 UTF-8，中文路径的保存文件复核已通过测试 [2]。

表 4: 捕获生命周期中的关键 API

API	用途及检查
pcap_findalldevs / freealldevs	获取并释放设备链表；遍历名称、描述、IPv4 与标志
pcap_create	创建未激活句柄，失败检查错误缓冲
pcap_set_snaplen	设置每帧捕获上限，默认 65535
pcap_set_promisc	默认关闭混杂，显式选项可开启
pcap_set_timeout / buffer_size	设置 100 ms 缓冲读取期限和 2 MiB 捕获缓冲
pcap_activate	激活设备；负数为错误，正数警告单独显示
pcap_datalink	仅允许 DLT_EN10MB 的以太网解析路径
pcap_compile / setfilter / freecode	可选 BPF 编译、安装与释放；过滤失败不能默默继续
pcap_setnonblock	设为非阻塞，确保空闲时仍能检测停止条件
pcap_next_ex	读取下一帧，区分成功、暂时无帧、错误和离线结束
pcap_stats	读取本句柄的捕获统计，失败不伪造零值
pcap_close	释放捕获句柄，正常与异常路径均执行

4.3 整体流程与退出机制

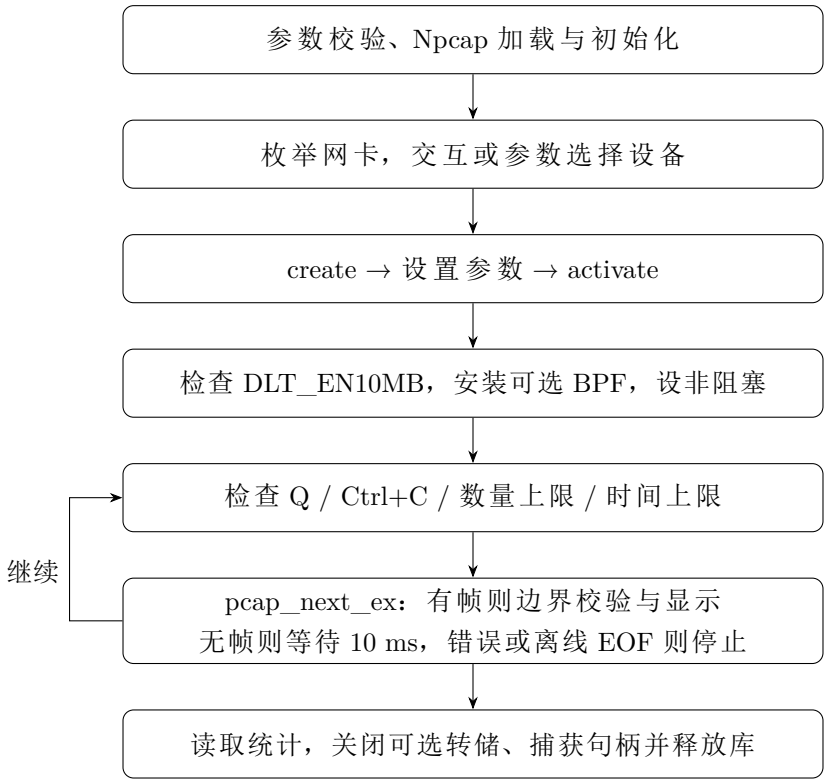


图 2: 实时捕获的主要控制流程；离线分支直接打开文件后进入相同解析路径

程序不能假定读取缓冲期限会在没有任何流量时定时唤醒阻塞读取，因此主动使用非阻塞模式 [3]。收到状态 1 时立即处理并打印该帧；状态 0 表示暂时没有数据，等待 10 ms 后重新检查停止条件；状态 -1 为错误，-2 表示离线读取结束或 break 情况 [4]。驱动返回的头部与数据指针只在后续读取前有效，程序在本轮完成解析和可选转储，不在下一轮继续引用旧指针。

以下是接收分支的简化核心片段，完整实现还记录计数、判断退出条件和处理可选保存：

```
const int status = api.pcap_next_ex(handle, &header, &packet);
if (status == 1) {
    print_frame(++received, *header, packet, json);
} else if (status == 0) {
    std::this_thread::sleep_for(std::chrono::milliseconds(10));
} else if (status == PCAP_ERROR_BREAK) {
    break;
} else {
    throw std::runtime_error(api.pcap_geterr(handle));
}
```

Q 不需要按回车；Ctrl+C 与 Ctrl+Break 通过控制台处理函数仅设置停止标志，资源由主流程释放。默认最多显示 20 帧或运行 30 秒；两项均可设为零取消限制。时间使用单调时钟，避免系统时钟调整干扰结束判断。

4.4 输出设计与可选保存

人类可读主表固定显示 `SRC-MAC`、`DST-MAC`、`Type` 和 `After`，并附时间、捕获/原始长度、协议名称。额外解释单独放在下一行；VLAN 内层解释不会覆盖主表原始字段。`--json` 提供同一组信息的 JSON Lines，供自动化检查，不改变捕获和解析逻辑。

默认仅在屏幕显示，不打开抓包文件。只有明确提供 `--save` 时才调用 `pcap_dump_open/pcap_dump` 保存原始帧。`--read` 使用 `pcap_open_offline` 复核已有文件，输出明确标为“离线复核”，不冒充实时捕获。设备列表、捕获句柄和可选转储分别由小型 RAII 资源管理器负责释放；Npcap 驱动、DLL 和第三方安装器不打包进提交文件。

5 实验步骤与测试验证

5.1 构建与实时操作

运行环境先安装 Npcap；重新编译时另需官方 SDK 和 Visual Studio C++ Build Tools。在仓库根目录执行：

```
.\lab02\bootstrap-sdk.ps1
.\lab02\build.ps1
.\lab02\bin\packet_capture.exe --list
.\lab02\bin\packet_capture.exe
```

最后一条会询问网卡编号。也可显式指定设备、过滤器、数量与时间；以下 6 只是本次枚举编号，不能假定在别的电脑上仍是同一网卡：

```
.\lab02\bin\packet_capture.exe --device 6 --count 6 --seconds 30 --filter "icmp"
ping -n 3 10.130.0.1
```

两条命令在不同终端运行。第二条产生流量；实际演示时应使用现场默认网关，避免照抄本机地址。

5.2 纯解析器测试

24 项 C++ 测试包含：空帧，1–13 字节短帧，恰好 14 字节，MAC 顺序，IPv4/IPv6/ARP 首字节，网络字节序，长度 0 和 1500，保留边界 1501 和 1535，EtherType 起点 1536，单/双 VLAN，截断 VLAN，无负载标签，snaplen 截断，广播 MAC，IPv4 地址、ARP 操作码、全部前缀、重复标签与未对齐缓冲。这些是明确的构造数据，用于验证边界；真实帧分析单独使用物理网卡捕获。

5.3 真实 exe 的集成测试

表 5: 集成测试覆盖范围与结果

编号	验证内容	结果
01	Npcap 实际版本、真实设备枚举、IPv4 与物理网卡匹配	通过
02	合成 PCAP：短帧、零首字节、VLAN 原始与内层偏移	通过
03	DLT_NULL 文件明确拒绝，不输出伪 MAC	通过
04	非法参数、负数、缺值与不存在的设备	通过
05	非法 BPF 与不存在的输入文件	通过
06	无匹配流量时，仍在 1 秒时限后主动退出	通过
07	实际 ICMP 捕获与 PCAP 原始字段逐条比较	通过
08	同一真实捕获文件在中文路径下读取、过滤	通过
09	数量达到 1 帧后准确退出	通过
10	空闲捕获进程收到 Ctrl+Break 后正常清理退出	通过
11	未指定 save 时不生成或改写抓包文件	通过

记录时间为 2026-10-22 00:37:12 +0800；共 24 项解析测试和 11 项集成测试全部通过，集成驱动总耗时 6.797 秒。CMake Release 构建和 CTest 解析测试也通过。完整日志与结构化结果分别保存在 evidence 目录的 `parser-tests.txt`、`integration-tests.txt` 和 `results.json`。

6 真实捕获结果与逐帧分析

6.1 捕获配置与总体结果

在实际 Wi-Fi 网卡上设置 `icmpandhost10.130.0.1`，只捕获与本机网关有关的 ICMP。默认关闭混杂模式，捕获上限 65535 字节，持续约 4.012 秒；另一个进程对网关发送三次 32 字节数据的 ping。共捕获 6 帧，分别为 3 个 Echo request 和 3 个 Echo reply。原始 PCAP 为 `evidence/real-capture.pcap`，实时程序输出为 `evidence/live-capture.jsonl`。

表 6: 本次真实捕获的统计

指标	实测值
应用读取帧数	6
短帧 / 截断帧	0 / 0
Npcap ps_recv	6
Npcap ps_drop	0
Npcap ps_ifdrop	0

`ps_drop` 表示捕获机制因缓冲等原因丢弃的计数；其值为零表示本次句柄没有报告此类捕获丢弃。`ps_ifdrop` 在不同平台可能不受支持，不能把零值当成整个物理网络无丢包的证明 [5]。三次 ping 均得到应答，是此次请求/应答的观察事实，不能外推为所有网络业务的丢包率。

6.2 题目必显字段的逐字节核对

下表从实时 JSON 输出生成；测试另行直接读取 PCAP 记录，比较目的 MAC 的 0–5 字节、源 MAC 的 6–11 字节、12–13 字节的大端值、偏移 14 及捕获/原始长度，六帧全部一致。

表 7: 真实六帧的必显字段；长度为 caplen/len

序号	源 MAC	目的 MAC	类型	首字节	长度
1	F8:59:E3:E3:B3:7E	00:00:5E:00:01:FE	0x0800	0x45	74/74
2	00:00:5E:00:01:FE	F8:59:E3:E3:B3:7E	0x0800	0x45	74/74
3	F8:59:E3:E3:B3:7E	00:00:5E:00:01:FE	0x0800	0x45	74/74
4	00:00:5E:00:01:FE	F8:59:E3:E3:B3:7E	0x0800	0x45	74/74
5	F8:59:E3:E3:B3:7E	00:00:5E:00:01:FE	0x0800	0x45	74/74
6	00:00:5E:00:01:FE	F8:59:E3:E3:B3:7E	0x0800	0x45	74/74

请求方向源 MAC 为 `F8:59:E3:E3:B3:7E`、目的 MAC 为 `00:00:5E:00:01:FE`；应答方向二者互换。它们是本次捕获的本地链路地址，不能仅根据目的 IP 就假定所有网络跳点都使用相同 MAC。类型字段均为 `0x0800`，表示后续为 IPv4。字段后首字节均为 `0x45`，高四位 4 是版本，低四位 5 表示 IPv4 头长为 $5 \times 4 = 20$ 字节。

6.3 第一帧的十六进制分析

以下数据直接从第一条真实 PCAP 记录读取，按原始偏移排版，不是手工编造的报文。

```
0000  00 00 5E 00 01 FE F8 59 E3 E3 B3 7E 08 00 45 00
0010  00 3C 43 C4 00 00 80 01 C0 C2 0A 82 21 36 0A 82
0020  00 01 08 00 4D 5A 00 01 00 01 61 62 63 64 65 66
0030  67 68 69 6A 6B 6C 6D 6E 6F 70 71 72 73 74 75 76
0040  77 61 62 63 64 65 66 67 68 69
```

偏移 0000 起六字节为目的 MAC，随后六字节为源 MAC；偏移 12–13 是 0800，偏移 14 是 45。IPv4 协议字段为 1，表示 ICMP；Echo request 的 ICMP type 为 8、code 为 0。本次每帧 74 字节，可分解为

$$14(\text{以太网头}) + 20(\text{IPv4 头}) + 8(\text{ICMP 头}) + 32(\text{测试数据}) = 74.$$

不能额外把前导码、SFD 或 FCS 加进本次捕获长度；Npcap 提供的是驱动交付的帧视图，FCS 通常不会出现在该缓冲中。

6.4 请求与应答配对

根据 IP 方向、ICMP 类型、标识符和序号配对；实际解析结果如下。

表 8: 六个真实帧的网络层及 ICMP 解释

序号	源 IPv4	目的 IPv4	ICMP 含义	序号字段
1	10.130.33.54	10.130.0.1	Echo request (8)	1
2	10.130.0.1	10.130.33.54	Echo reply (0)	1
3	10.130.33.54	10.130.0.1	Echo request (8)	2
4	10.130.0.1	10.130.33.54	Echo reply (0)	2
5	10.130.33.54	10.130.0.1	Echo request (8)	3
6	10.130.0.1	10.130.33.54	Echo reply (0)	3

请求的源 IP 为本机 10.130.33.54，目的为默认网关 10.130.0.1，应答反向；应答 type 为 0，code 为 0。逐帧结构化解析另存于 evidence/frame-analysis.json，其中还包含原始 PCAP 的 SHA256，便于确认表格对应的文件。

6.5 程序界面输出

以下节选同一份真实 PCAP 在程序中离线复核的前两帧标准输出，明确显示“离线复核”；完整屏幕输出与实时 JSON 记录已保存于 evidence 目录。这展示了界面的必显列、时间、长度和协议解释。离线读取耗时仅代表读取该文件的时间，不等于上面的真实捕获持续时间。

```
Npcap version 1.89, based on libpcap version 1.10.7 (64-bit time_t)
离线复核已开始，按 Q 或 Ctrl+C 退出。Type 字段为偏移 12-13，After 为偏移 14。
 No. Time            SRC-MAC           DST-MAC           Type   After Cap/Wire Protocol
   1 00:37:07.225884 F8:59:E3:E3:B3:7E 00:00:5E:00:01:FE 0x0800 0x45 74/74 IPv4
     10.130.33.54 -> 10.130.0.1 IP.protocol=1
   2 00:37:07.228329 00:00:5E:00:01:FE F8:59:E3:E3:B3:7E 0x0800 0x45 74/74 IPv4
     10.130.0.1 -> 10.130.33.54 IP.protocol=1
```

7 问题处理、结论与局限

7.1 主要问题及解决

1. **驱动存在却无法加载 API**。原环境缺少配套 DLL。完成官方 Npcap 安装后，以实际 API 调用和物理网卡捕获验证，而不是仅查驱动文件是否存在。
2. **回环并非 Ethernet**。捕获之前检查链路类型；非以太网句柄明确拒绝，避免显示没有意义的“MAC”。
3. **类型字段与后首字节易混淆**。固定主表的原始偏移，VLAN 内层字段单独展示；以合成测试验证零值、双标签及截断。
4. **空闲时阻塞导致无法退出**。使用非阻塞 next_ex 与短暂等待，持续检查数量、时间及退出标志；无流量和 Ctrl+Break 测试均通过。
5. **中文路径受系统代码页影响**。宽字符入口和 pcap_init UTF-8 模式共同处理，已用中文文件名验证同一真实捕获可重放。
6. **屏幕输出需要独立核对**。将可选保存的真实帧与 JSON 字段逐项比较，避免仅看“程序有输出”就判断解析正确。

7.2 实验结论

本实验完成 Npcap 网卡枚举、打开、实时捕获、字段显示与正常退出。使用 C++ 直接处理捕获字节，24 项解析测试和 11 项集成测试全部通过；实际六个 ICMP 帧的源/目的 MAC、类型字段、字段后首字节与原始 PCAP 完全一致。默认只输出屏幕，持久化是可选复核功能；交付含源码、可执行文件和使用本人南开模板完成的 LaTeX 报告。

局限是仅对 DLT_EN10MB 作以太网字段解析，非以太网类型明确拒绝；实际样本来自一张 Wi-Fi 网卡和一次本机网关测试，VLAN/ARP/IPv6 等边界覆盖不能冒充本次真实网卡观测。统计只反映该捕获句柄和本次测试，不证明物理网络永不丢包。现场运行仍需安装 Npcap，并按实际网卡编号和网络流量选择设备。

参考文献

[1] Nmap Project. Developing software with Npcap. https://npcap.com/guide/npcap-devguide.html.

[2] Nmap Project. pcap_init. https://npcap.com/guide/wpcap/pcap_init.html.

[3] Nmap Project. pcap_setnonblock. https://npcap.com/guide/wpcap/pcap_setnonblock.html.

[4] Nmap Project. pcap_next_ex. https://npcap.com/guide/wpcap/pcap_next_ex.html.

[5] Nmap Project. pcap_stats. https://npcap.com/guide/wpcap/pcap_stats.html.

[6] IANA. IEEE 802 Numbers. https://www.iana.org/assignments/ieee-802-numbers/.